

Financial Copilot Bank Statement & Cashflow Intelligence Blueprint

AI-agent implementation brief for bank statement uploads, income detection, expenditure analysis, categorisation, reporting, and cashflow recommendations.

Purpose: Enhance the Financial Copilot platform so users can upload bank statements and the system can analyse income, expenses, spending behaviour, cashflow, subscriptions, affordability, savings capacity, and investment funding ability.

Recommended approach: Build this as an ingestion and analytics module first, then expose deterministic cashflow tools to the AI orchestrator. The LLM must explain results, not invent them.

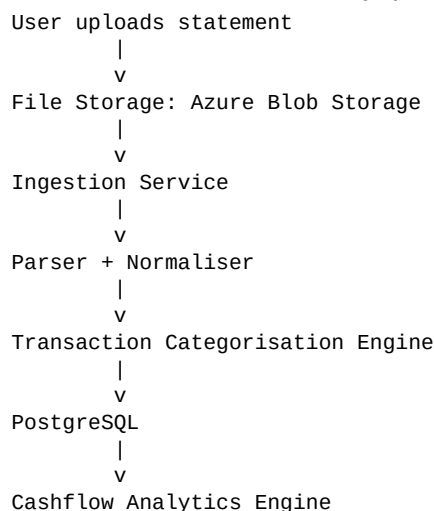
1. Product Outcome

The feature must allow a user to upload bank statements and receive a clear monthly cashflow report. The system must detect income, recurring expenses, discretionary spending, debt payments, savings transfers, investment contributions, subscriptions, abnormal transactions, and month-on-month trends.

- Upload PDF, CSV, or Excel bank statements.
- Parse transactions into a standard transaction schema.
- Automatically categorise transactions.
- Detect salary or business income.
- Detect recurring expenses and subscriptions.
- Calculate monthly cashflow surplus or deficit.
- Calculate savings and investment rate.
- Highlight spending leaks.
- Produce monthly reports.
- Allow the AI assistant to answer cashflow questions using verified calculations.

2. Recommended Architecture

Use the Spring Boot API as the system of record. Use a separate ingestion pipeline for parsing and normalisation. The Python AI orchestrator should only query validated cashflow data through backend APIs.



|
v
Dashboard + AI Orchestrator

3. Repository Responsibilities

Repo	Responsibility
wealthpilot-api	Bank transaction storage, categorisation, cashflow calculations, reporting APIs
wealthpilot-web	Statement upload UI, cashflow dashboard, spending reports, correction screens
wealthpilot-ai-orchestrator	Explains cashflow insights using backend tools, generates narrative reports
wealthpilot-infra	Blob Storage, PostgreSQL, Container Apps, Key Vault, monitoring, CI/CD

4. Data Sources to Support

Build the ingestion system to support multiple formats because banks differ significantly.

Source	Priority	Notes
CSV export	High	Best for reliable parsing
PDF statement	High	Common user upload format
Excel export	Medium	Useful for manual users
Bank API/Open Banking	Later	Production enhancement
Manual entry	Medium	Fallback when parsing fails

5. Standard Transaction Model

Every imported bank transaction must be converted into one clean internal format.

```
bank_transactions
- id
- user_id
- account_id
- source_file_id
- transaction_date
- description_raw
- description_clean
- money_in
- money_out
- amount_signed
- balance_after_transaction
- currency
- counterparty
- category_id
- subcategory_id
- transaction_type
- is_recurring
- recurrence_group_id
- confidence_score
- user_corrected boolean
- duplicate_hash
- created_at
- updated_at
```

6. Categorisation Model

Use deterministic rules first, then ML/AI assistance. User corrections must become training data for future categorisation.

Category	Examples
Income	Salary, side hustle, rental income, dividends
Housing	Bond, rent, levies, rates, insurance
Utilities	Electricity, water, internet, mobile data
Groceries	Checkers, Pick n Pay, Woolworths food
Transport	Fuel, Gautrain, Uber, car finance
Insurance	Life cover, car insurance, medical aid
Debt	Credit card, personal loan, vehicle finance
Investments	EasyEquities, RA, TFSA, brokerage transfers
Subscriptions	Netflix, Spotify, ChatGPT, Microsoft, Apple
Discretionary	Restaurants, clothing, entertainment
Transfers	Internal transfers, savings movements
Unknown	Needs user review

7. Categorisation Engine Rules

Build a layered categorisation engine.

- Layer 1: Exact user rules
- Layer 2: Merchant keyword rules
- Layer 3: Recurring transaction matching
- Layer 4: Bank reference pattern matching
- Layer 5: AI-assisted suggestion
- Layer 6: User review and correction

Example rules

```
IF description contains 'SALARY' OR matches employer name:
    category = INCOME
    transaction_type = SALARY

IF description contains 'EASYEQUITIES':
    category = INVESTMENTS

IF description contains 'ABSA HOME LOAN' OR 'BOND':
    category = HOUSING
    transaction_type = BOND_PAYMENT

IF same merchant, similar amount, monthly interval:
    mark as recurring
```

8. Income Detection

The system must distinguish between salary, business income, transfers, refunds, and once-off deposits.

- Detect recurring salary deposits by amount, date, and description.
- Detect side-hustle or business income separately.
- Exclude internal transfers from true income.
- Flag irregular once-off inflows.

- Calculate stable monthly income and average monthly income.
- Support multiple income streams.

9. Expense Analysis

The cashflow engine must analyse spending by month, category, merchant, frequency, and essential vs discretionary nature.

- Fixed expenses: bond, insurance, subscriptions, school fees, medical aid.
- Variable essentials: groceries, utilities, fuel.
- Discretionary expenses: restaurants, entertainment, clothing, gadgets.
- Debt payments: credit cards, loans, vehicle finance.
- Investment contributions: TFSA, RA, ETFs, USD transfers.
- Hidden leaks: small recurring purchases and unused subscriptions.

10. Cashflow Metrics

The analytics engine must produce these monthly metrics.

Total Income
Total Expenses
Essential Expenses
Discretionary Expenses
Debt Payments
Investment Contributions
Savings Contributions
Net Cashflow
Cashflow Margin
Savings Rate
Investment Rate
Debt Service Ratio
Housing Cost Ratio
Subscription Burden
Emergency Fund Coverage
Lifestyle Inflation Trend

11. Financial Intelligence Reports

Generate reports that are factual, concise, and useful.

Report	Purpose
Monthly Cashflow Report	Shows income, expenses, surplus, trends
Spending Breakdown	Shows category and merchant spending
Subscriptions Report	Finds recurring digital and service costs
Affordability Report	Shows free cashflow for investing or bond overpayment
Investment Funding Report	Shows how much the user can safely invest
Debt Pressure Report	Shows debt payments relative to income
Behavioural Report	Highlights spending patterns and lifestyle creep

12. Dashboard Requirements

- Cashflow summary cards: income, expenses, surplus, savings rate.

- Spending by category chart.
- Month-on-month trend chart.
- Recurring expense list.
- Top merchants by spend.
- Income reliability indicator.
- Investment contribution tracker.
- Debt payment tracker.
- Alerts for unusual spending.
- Manual category correction table.

13. AI Orchestrator Tooling

Expose deterministic tools to the AI orchestrator. The AI must call tools and explain results.

```
get_monthly_cashflow(userId, month)
get_cashflow_trends(userId, startMonth, endMonth)
get_spending_by_category(userId, month)
get_recurring_expenses(userId)
get_income_summary(userId)
get_investment_contributions(userId, month)
get_affordability_summary(userId)
find_spending_leaks(userId)
generate_cashflow_report(userId, month)
```

AI assistant rules

- Never guess income or expenses.
- Always use stored transaction data.
- Explain assumptions clearly.
- Separate transfers from income.
- Separate investments from expenses.
- Avoid shame-based language.
- Offer options, not commands.
- Ask for user confirmation when categorisation confidence is low.

14. User Correction Workflow

Categorisation will never be perfect. Build correction as a first-class workflow.

- Show low-confidence transactions to the user.
- Allow the user to change category and subcategory.
- Allow the user to mark transaction as internal transfer.
- Allow the user to mark transaction as recurring.
- Save correction rules for future imports.
- Show confidence score and reason for category selection.

15. Duplicate Detection

Avoid double-counting transactions when a user uploads overlapping statements.

```
duplicate_hash = hash(
    account_id +
    transaction_date +
    amount_signed +
    normalised_description +
    balance_after_transaction
)
```

```
If duplicate_hash already exists:
    skip or flag as duplicate
```

16. Security and Privacy Requirements

Bank statements contain highly sensitive data. Implement strict security from the start.

- Store uploaded files in Azure Blob Storage with private access only.
- Encrypt sensitive fields at rest.
- Use Azure Key Vault for secrets.
- Never expose bank statement files publicly.
- Do not send raw full bank statements to the LLM unless explicitly required.
- Prefer sending summaries and structured data to the AI.
- Log access to statement data.
- Allow users to delete uploaded statements and parsed transactions.
- Mask account numbers in UI.
- Apply least-privilege access controls.

17. Implementation Phases

Phase	Build	Deliverable
1	Manual cashflow model	User can enter income/expense manually
2	CSV upload	Parse bank CSV into standard transactions
3	PDF upload	Parse statement PDFs and extract transactions
4	Categorisation	Rule engine with user corrections
5	Cashflow dashboard	Income, expenses, surplus, trends
6	Recurring detection	Subscriptions and fixed costs
7	AI reporting	AI explains cashflow using backend tools
8	Affordability engine	Investment and bond overpayment capacity
9	Advanced intelligence	Lifestyle creep, anomaly detection, forecasts
10	Bank integrations	Optional Open Banking/API connections

18. Acceptance Criteria

- User can upload a bank statement file.
- Transactions are parsed into a standard schema.

- Income and expenses are separated correctly.
- Internal transfers are not counted as income.
- Investments are recognised separately from lifestyle spending.
- Recurring expenses are detected.
- User can correct categories.
- Duplicate imports are handled.
- Dashboard shows monthly cashflow accurately.
- AI assistant explains cashflow using backend data.
- No raw bank data is exposed to the frontend unnecessarily.
- Tests cover parsers, categorisation rules, duplicate detection, and calculations.

19. Recommended Prompt to Give the Coding Agent

Implement the Bank Statement and Cashflow Intelligence module.

Follow this blueprint exactly.

Build incrementally:

1. Database migrations
2. Backend transaction model
3. File upload endpoint
4. CSV parser
5. PDF parser interface
6. Normalisation service
7. Categorisation rule engine
8. Duplicate detection
9. Cashflow analytics service
10. Dashboard APIs
11. Frontend upload and dashboard screens
12. AI orchestrator tools
13. AI cashflow report generation
14. Tests and sample data

Do not let the LLM calculate cashflow directly.

All calculations must come from deterministic backend services.

The AI layer must only explain and interpret the verified results.

20. Strategic Product Note

This module is commercially important. Once the platform understands bank statements, it moves beyond investment tracking into personal CFO territory. It can advise on affordability, overspending, savings capacity, debt pressure, and how much the user can responsibly allocate to TFSA, RA, USD ETFs, ZAR ETFs, and bond overpayments.

End of blueprint. This document is ready to upload into Cursor, ChatGPT Agent, Devin, or another AI coding agent.